БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

ФАКУЛЬТЕТ РАДИОФИЗИКИ И КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ

Кафедра информатики и компьютерных систем

ДИПЛОМНАЯ РАБОТА

«Разработка комплекса средств для анализа вычислительных возможностей микропроцессорных систем»

Пашука Тимофея Андреевича
обучающегося 4 курса специальности
«Компьютерная безопасность»

Научный руководитель:
Старший преподаватель
Дрозд Александр Николаевич

Минск, 2026

Оглавление

Введение **3**

1 Анализ предметной области **7**

1.1 Архитектурные основы современных ЦП 7

1.2 Иерархия памяти в современных вычислительных системах . . 8

1.3 Модели параллелизма и конкурентности 9

2 Обзор существующих средств анализа и профилирования **11**

3 Проектирование комплекса средств анализа вычислительных возможностей **13**

3.1 Формулирование требований к программному комплексу 13

3.1.1 Функциональные требования 13

3.1.2 Нефункциональные требования 15

3.2 Обоснование выбора технологического стека 15

3.2.1 Выбор языка программирования 15

3.2.2 Выбор библиотеки для GUI 16

3.3 Проектирование архитектуры 17

3.3.1 Высокоуровневая структура 17

3.3.2 Проектирование основных модулей 17

4 Реализация и тестирование программного комплекса **19**

4.1 Реализация пользовательского интерфейса 19

4.1.1 Панель мониторинга 19

4.1.2 Панели тестирования 20

4.2 Реализация и результаты тестов 21

4.2.1 Тесты производительности ЦП 21

4.2.2 Тесты иерархии памяти 22
4.2.3 Тесты механизмов параллелизма 23

Заключение **25**

Список использованных источников **28**

Приложение А. Листинг основного файла main.go **31**

Приложение Б. Листинг обработчика тестов runner.go **33**

Приложение В. Листинг теста задержки памяти latency.go **35**

Приложение Г. Листинг теста горутин goroutines.go **37**

Введение

Актуальность темы

Эволюция вычислительной техники за последние десятилетия прошла путь от простых одноядерных процессоров до сложнейших многоядерных систем на кристалле (System-on-a-Chip, SoC), включающих в себя гетерогенные архитектуры с производительными (Performance-cores) и энергоэффективными (Efficiency-cores) ядрами, многоуровневые иерархии кэш-памяти, векторные расширения и специализированные сопроцессоры. Закон Мура, гласивший об удвоении числа транзисторов на кристалле каждые два года, привел к экспоненциальному росту вычислительной мощности. Однако с исчерпанием возможностей по наращиванию тактовой частоты (проблема «стены мощности»), фокус сместился на увеличение числа ядер и усложнение архитектуры.

В этих условиях эффективность программного обеспечения перестала быть прямо пропорциональной «сырой» производительности процессора. Теперь она напрямую зависит от того, насколько хорошо программа способна утилизировать все доступные аппаратные ресурсы: распараллеливать вычисления между ядрами, эффективно использовать кэш-память для минимизации задержек и задействовать специализированные инструкции. Для разработчиков, системных архитекторов и даже конечных пользователей понимание реальных возможностей и «узких мест» конкретной аппаратной платформы становится критически важной задачей.

Существующий на рынке инструментарий для анализа производительности можно условно разделить на две категории. С одной стороны, это профессиональные, узкоспециализированные утилиты (например, Intel VTune Profiler, AMD uProf), которые предоставляют исчерпывающую информацию, но требуют глубоких знаний в области архитектуры ЭВМ и сложны в освоении. С другой стороны, это простые системные мониторы (встроенные в ОС

«Диспетчер задач» или «Мониторинг системы»), которые отображают лишь общую информацию о загрузке, не позволяя провести углубленный синтетический анализ. Образовался разрыв: отсутствует доступный и простой в использовании инструмент, который бы сочетал в себе наглядность системного монитора и глубину синтетических тестов.

Данная работа направлена на заполнение этой ниши путем создания интегрированного программного комплекса, который предоставляет пользователю удобный графический интерфейс для одновременного мониторинга системных ресурсов и проведения набора стандартизированных тестов (бенчмарков) для ключевых компонентов системы. Актуальность такой разработки определяется потребностью в универсальном, кросс-платформенном и наглядном инструменте для оценки и сравнения производительности современных микропроцессорных систем.

Цели и задачи исследования

Целью данной дипломной работы является проектирование и разработка кросс-платформенного программного комплекса для углубленного анализа производительности и системного мониторинга. Работа включает в себя детальное исследование теоретических основ, таких как архитектура современных ЦП и иерархия памяти, проектирование отказоустойчивой и расширяемой архитектуры, практическую реализацию на языке Go с использованием библиотеки Fyne, а также проведение, подробный анализ и интерпретацию результатов серии синтетических тестов для всех ключевых подсистем компьютера. Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Провести углубленный анализ теоретических основ оценки производительности вычислительных систем, изучить классификацию бенчмарков и методологию их проведения.

2. Выполнить сравнительный анализ существующих технологий и программных средств для реализации поставленной задачи, на основе которого обосновать выбор технологического стека (язык Go, библиотека Fyne).

3. Спроектировать модульную архитектуру приложения, обеспечивающую слабую связанность компонентов, их переиспользуемость и возможность дальнейшего расширения функционала.

4. Реализовать модуль для мониторинга основных параметров системы в реальном времени: общей и по-ядерной загрузки ЦП, использования физической и виртуальной памяти, скорости дисковых и сетевых операций, а также общей информации о системе.

5. Разработать и реализовать набор синтетических тестов для оценки производительности:

   - Центрального процессора (целочисленные и вещественнозначные операции).
   - Подсистемы памяти (измерение задержки доступа к кэшам L1, L2, L3 и ОЗУ; измерение пропускной способности ОЗУ и постоянных накопителей).
   - Механизмов параллельной обработки (накладные расходы на создание горутин, производительность примитивов синхронизации).

6. Создать кросс-платформенный графический пользовательский интерфейс для наглядной визуализации данных мониторинга и интерактивного управления процессом тестирования.

7. Провести экспериментальное тестирование разработанного комплекса на различных аппаратных платформах, проанализировать полученные результаты и продемонстрировать практическую применимость инструмента.

Объект, предмет и структура работы

Объектом исследования является процесс анализа и оценки производительности современных микропроцессорных систем с гетерогенной архитектурой.

Предметом исследования выступает программный комплекс, реализующий функции мониторинга и бенчмаркинга системных ресурсов, и методология его применения для анализа вычислительных возможностей.

Структура работы. Дипломная работа состоит из введения, двух основных глав, заключения, списка использованных источников и приложений. Во введении обосновывается актуальность темы, ставятся цели и задачи. В первой, теоретической, главе рассматриваются методология анализа производительности и дается обоснование выбора средств реализации. Во второй, практической, главе подробно описывается архитектура и реализация разработанного программного комплекса, а также проводится анализ результатов его работы. В заключении подводятся итоги и намечаются пути дальнейшего развития проекта. Приложения содержат листинги ключевых фрагментов исходного кода.

Глава 1

Анализ предметной области

1.1 Архитектурные основы современных ЦП

Современные центральные процессоры (ЦП) — это сложные системы, производительность которых определяется не только тактовой частотой, но и множеством архитектурных решений, направленных на ускорение выполнения инструкций. Ключевыми из них являются конвейеризация и суперскалярное исполнение.

Конвейер инструкций (Instruction Pipeline). Это технология, при которой обработка инструкции разбивается на несколько последовательных стадий (например: выборка, декодирование, исполнение, доступ к памяти, запись результата). Несколько инструкций могут находиться на разных стадиях конвейера одновременно. Это позволяет процессору завершать в среднем одну инструкцию за такт, даже если полная обработка каждой инструкции занимает несколько тактов. Тесты целочисленной и вещественной арифметики в данной работе напрямую измеряют эффективность этого конвейера.

Суперскалярная архитектура. Это развитие идеи конвейера, при котором процессор имеет несколько параллельных исполнительных блоков (например, несколько ALU для целочисленных операций и несколько FPU для операций с плавающей запятой). Это позволяет ему выполнять несколько инструкций одновременно на одной и той же стадии конвейера, если между ними нет зависимостей по данным. Современные процессоры могут исполнять от 4 до 8 инструкций за такт.

Гетерогенные архитектуры. Современной тенденцией, особенно в мобильных и ARM-процессорах (как Apple M1), является использование гетерогенных ядер: высокопроизводительных (Performance-cores, P-cores) и энергоэффективных (Efficiency-cores, E-cores). P-cores имеют более сложную архитектуру, больше кэша и выше частоты, обеспечивая максимальную производительность. E-cores проще, меньше и потребляют значительно меньше энергии, идеально подходя для фоновых задач. Разработанное приложение способно определять и отображать нагрузку на эти типы ядер раздельно.

1.2 Иерархия памяти в современных вычислительных системах

Производительность современных компьютеров определяется не только скоростью процессора, но и эффективностью работы подсистемы памяти. Процессоры работают на частотах, на порядки превышающих скорость доступа к основной оперативной памяти (DRAM). Чтобы сгладить эту разницу, используется многоуровневая иерархия кэш-памяти.

- **Регистры процессора:** Наиболее скоростной тип памяти, интегрированный непосредственно в ядро процессора. Время доступа составляет 1 такт.

- **Кэш-память (L1, L2, L3):** Строится на основе статической памяти (SRAM), которая очень быстра, но дорога и энергозатратна. SRAM не требует регенерации и хранит данные, пока подается питание. Именно поэтому объемы кэш-памяти невелики.

- **Оперативная память (DRAM):** Строится на основе динамической памяти, где каждый бит хранится в виде заряда в конденсаторе. Конденсаторы со временем разряжаются, поэтому память требует постоянной регенерации, что делает ее медленнее SRAM. Однако она значительно дешевле и компактнее, что позволяет создавать модули большого объема.

- **Постоянные накопители (SSD/HDD):** Хранят данные постоянно. Время доступа на порядки медленнее ОЗУ.

Эффективность работы этой иерархии основана на **принципе локальности**, который гласит, что программы имеют тенденцию повторно использовать данные и инструкции, к которым они недавно обращались (временная локальность), а также обращаться к данным, расположенным в памяти по соседству (пространственная локальность). Понимание задержек и пропускной способности каждого уровня иерархии является ключевым для написания производительного кода.

1.3 Модели параллелизма и конкурентности

С ростом числа ядер в процессорах параллельное программирование стало стандартом. Существует несколько моделей организации параллельных вычислений:

- **Многопоточность с общей памятью (Shared-memory multithreading):** Несколько потоков выполнения работают в одном адресном пространстве и имеют доступ к общим данным. Это наиболее распространённая модель в современных ОС. Основная сложность связана с синхронизацией доступа к общим данным и предотвращением состояний гонки. Для этого используются примитивы синхронизации (мьютексы, семафоры, атомарные операции).

- **Модель передачи сообщений (Message Passing):** Параллельные процессы имеют изолированную память и взаимодействуют исключительно путём обмена сообщениями. Ярким примером является стандарт MPI (Message Passing Interface), широко применяемый в высокопроизводительных вычислениях (HPC). Эта модель позволяет избежать многих проблем с синхронизацией, но вносит накладные расходы на сериализацию и пересылку данных.

- **Модель акторов:** Вариация модели передачи сообщений, в которой каждый «актор» представляет собой легковесный процесс со своим состоянием и взаимодействует с другими акторами только через обмен сообщениями. Язык Go с его горутинами и каналами реализует схожую философию: «Не обменивайтесь данными через общую память; обменивайтесь памятью через передачу данных».

Тестирование производительности примитивов синхронизации и механизмов коммуникации (каналов) позволяет оценить, насколько эффективно можно распараллелить задачу на данной системе с использованием конкретного языка программирования.

Глава 2

Обзор существующих средств анализа и профилирования

На рынке представлено множество инструментов для анализа производительности, каждый из которых ориентирован на свою специализированную область применения.

- **Системные утилиты ОС:** Такие как `top`, `htop`, `iostat` в Linux/macOS и «Диспетчер задач»/«Монитор ресурсов» в Windows. Они предоставляют высокоуровневую картину загрузки системы в реальном времени, но не позволяют проводить глубокий синтетический анализ или измерять специфические параметры, такие как задержка кэша.

- **Профилировщики от производителей ЦП:** Intel VTune Profiler, AMD uProf. Это мощнейшие инструменты, использующие доступ к аппаратным счетчикам производительности (Performance Monitoring Units, PMU). Они могут отслеживать такие события, как промахи в кэше, неверно предсказанные ветвления и т.д. Их главный недостаток — сложность и привязка к конкретному производителю.

- **Синтетические бенчмарки:** Наборы тестов, такие как Geekbench, Phoronix Test Suite, AIDA64. Они популярны для сравнения общей производительности систем, но часто представляют собой «черный ящик»,

не раскрывая методологию своих тестов и не позволяя анализировать отдельные низкоуровневые компоненты.

- **Инструменты профилирования в языках программирования:** Например, встроенный в Go профилировщик pprof. Он отлично подходит для анализа производительности конкретного Go-приложения (поиск «горячих» функций, анализ использования памяти), но не предназначен для анализа аппаратной платформы в целом.

Анализ показывает, что существует незанятая ниша для инструмента, который бы сочетал простоту и наглядность системных утилит с возможностью проведения понятных и воспроизводимых синтетических тестов для ключевых аппаратных компонентов, будучи при этом кросс-платформенным и не привязанным к конкретному производителю.

Глава 3

Проектирование комплекса средств анализа вычислительных возможностей

3.1 Формулирование требований к программному комплексу

Проектирование любого программного продукта начинается с определения набора требований, которые описывают, что система должна делать и какими свойствами обладать. Требования делятся на две основные категории: функциональные и нефункциональные.

3.1.1 Функциональные требования

Функциональные требования определяют конкретные функции, которые система должна предоставлять пользователю. На основе анализа предметной области были сформулированы следующие требования:

1. **Мониторинг системы в реальном времени.** Система должна непрерывно собирать и отображать следующие метрики:

    - Общую и по-ядерную загрузку центрального процессора.

- Информацию о модели ЦП, его частоте и количестве ядер.
- Объем использованной и доступной оперативной памяти.
- Текущую скорость операций чтения и записи на дисковых накопителях.
- Текущую скорость приема и передачи данных по сети.
- Общую информацию об операционной системе и времени ее работы (uptime).

2. **Проведение синтетических тестов ЦП.** Система должна предоставлять возможность запускать тесты для оценки производительности процессора, включая:
   - Тест на скорость выполнения целочисленных операций.
   - Тест на скорость выполнения операций с плавающей запятой.
3. **Проведение тестов подсистемы памяти.** Система должна включать бенчмарки для оценки различных аспектов иерархии памяти:
   - Тесты для измерения задержки (latency) доступа к кэшам L1, L2, L3 и к основной оперативной памяти.
   - Тесты для измерения пропускной способности (bandwidth) оперативной памяти при операциях чтения и записи.
   - Тесты для измерения производительности постоянных накопителей (SSD/HDD).
4. **Проведение тестов механизмов параллелизма.** Система должна оценивать эффективность встроенных в язык программирования средств для параллельной обработки данных.
5. **Графический пользовательский интерфейс.** Все функции мониторинга и тестирования должны быть доступны через единый, интуитивно понятный графический интерфейс. Интерфейс должен наглядно представлять данные и обеспечивать удобное управление.

3.1.2 Нефункциональные требования

Нефункциональные требования определяют атрибуты качества и ограничения для системы. Они не менее важны, чем функциональные, так как определяют, насколько хорошо система выполняет свои функции.

- **Кросс-платформенность.** Приложение должно корректно работать на основных операционных системах: Windows, macOS и Linux.
- **Низкие накладные расходы.** Программный комплекс, предназначенный для анализа производительности, сам должен оказывать минимальное влияние на измеряемую систему.
- **Отзывчивость интерфейса.** Графический интерфейс должен оставаться отзывчивым и не «зависать» во время выполнения длительных тестов или сбора данных.
- **Простота развертывания.** Приложение должно распространяться в виде единого исполняемого файла, не требующего от пользователя установки дополнительных зависимостей или библиотек.
- **Модульность и расширяемость.** Архитектура должна позволять легко добавлять новые тесты или модули мониторинга в будущем.

3.2 Обоснование выбора технологического стека

На основе сформулированных требований был проведен анализ и выбор технологического стека. Ключевыми компонентами стека являются язык программирования и библиотека для создания графического интерфейса.

3.2.1 Выбор языка программирования

Рассматривались следующие кандидаты: C++, Python, Rust и Go.

- **C++:** Традиционный выбор для высокопроизводительных системных утилит. Обеспечивает максимальную производительность и полный контроль над ресурсами. Однако его использование сопряжено с высокой

сложностью разработки, ручным управлением памятью и проблемами при обеспечении кросс-платформенной сборки GUI.

- **Python:** Популярен для быстрой разработки, имеет отличные библиотеки для анализа данных. Главный недостаток — низкая производительность интерпретатора и наличие глобальной блокировки интерпретатора (GIL), что затрудняет написание по-настоящему параллельного кода для утилизации всех ядер ЦП. Это делает его непригодным для написания точных бенчмарков.

- **Rust:** Современный язык, ориентированный на безопасность и производительность, сопоставимую с C++. Обладает сложной моделью владения и заимствования, что значительно повышает порог вхождения и замедляет разработку по сравнению с Go.

- **Go (Golang):** Выбран как оптимальный компромисс. Он предлагает производительность, близкую к C++ и Rust, но при этом значительно проще в освоении. Его модель параллелизма с горутинами и каналами идеально подходит для задач асинхронного обновления GUI и фонового выполнения тестов. Встроенная кросс-компиляция и статическая линковка полностью удовлетворяют требованиям к простоте развертывания.

3.2.2 Выбор библиотеки для GUI

Для языка Go существует несколько подходов к созданию GUI. Рассматривались Wails, Gio UI и Fyne.

- **Wails:** Позволяет использовать веб-технологии (HTML/CSS/JS) для создания интерфейса. Это дает большую гибкость в дизайне, но требует знаний веб-стека и может приводить к большему потреблению памяти и оверхеду из-за встроенного веб-движка.

- **Gio UI:** Низкоуровневая библиотека, использующая модель немедленного рендеринга (immediate mode). Предоставляет максимальную производительность и гибкость, но требует от разработчика написания большего количества кода для управления состоянием и отрисовкой.

- **Fyne:** Выбран как наиболее сбалансированное решение. Это чисто нативный инструментарий, написанный на Go, который не использует веб-технологии. Он предлагает декларативный подход (retained mode), богатый набор готовых виджетов и автоматическое соответствие внешнему виду целевой ОС. Это позволило быстро разработать функциональный и современно выглядящий интерфейс, полностью удовлетворив требования проекта.

3.3 Проектирование архитектуры

На основе требований и выбранного стека была спроектирована многоуровневая модульная архитектура.

3.3.1 Высокоуровневая структура

Приложение разделено на три основных слоя:

1. **Слой представления (GUI Layer):** Отвечает за отображение информации и взаимодействие с пользователем. Реализован в пакете `pkg/gui`.

2. **Слой бизнес-логики (Business Logic Layer):** Содержит логику выполнения бенчмарков и сбора системных метрик. Реализован в пакетах `pkg/benchmark`, `pkg/memory`, `pkg/concurrency` и `pkg/profiling`.

3. **Слой доступа к данным (Data Access Layer):** В данном проекте этот слой представлен библиотекой `gopsutil`, которая абстрагирует системные вызовы ОС.

Такое разделение позволяет каждому слою быть независимым. Например, тесты из слоя бизнес-логики можно запускать отдельно, без графического интерфейса, что упрощает их отладку.

3.3.2 Проектирование основных модулей

Модуль GUI и принципы UX. При проектировании интерфейса основное внимание уделялось пользовательскому опыту (UX). Главный принцип

— *неблокирующий интерфейс.* Любая операция, которая может занять больше нескольких миллисекунд (сбор данных, запуск теста), выполняется в отдельной горутине. Это гарантирует, что пользователь всегда может взаимодействовать с приложением: переключать вкладки, прокручивать содержимое, изменять размер окна. Второй принцип — *наглядная обратная связь.* Во время выполнения тестов пользователь видит не статичный экран, а два прогресс-бара: индивидуальный для текущего теста и общий для всей серии. Это дает четкое понимание, на какой стадии находится процесс и сколько примерно осталось ждать.

Модуль запуска тестов ('TestRunner'). Для унификации процесса тестирования спроектирован универсальный обработчик `benchmark.TestRunner`, являющийся центральным элементом логики бенчмаркинга. Его архитектура решает несколько задач:

- **Абстрагирование логики теста.** 'TestRunner' принимает на вход функцию любого теста. Ему не важна внутренняя реализация бенчмарка, главное, чтобы функция возвращала результат и единицу измерения. Это позволяет легко добавлять новые тесты, не изменяя логику их выполнения.

- **Статистическая достоверность.** 'TestRunner' выполняет каждый тест несколько раз (по умолчанию 5). Затем он отбрасывает крайние значения (минимальное и максимальное) и усредняет оставшиеся. Это позволяет получить более стабильный и достоверный результат, отсеяв случайные выбросы, вызванные фоновой активностью ОС.

- **Асинхронное уведомление о прогрессе.** Для обеспечения обратной связи в GUI `TestRunner` принимает на вход канал Go (`chan float64`). В ходе выполнения итераций теста он отправляет в этот канал значения от 0.0 до 1.0, сигнализируя о прогрессе. GUI-панель, запустившая тест, считывает данные из этого канала и обновляет соответствующий прогресс-бар. Использование каналов — идиоматичный и безопасный способ межгорутинного взаимодействия в языке Go.

Глава 4

Реализация и тестирование программного комплекса

4.1 Реализация пользовательского интерфейса

Пользовательский интерфейс (GUI) является точкой входа для взаимодействия пользователя с системой. Он спроектирован с упором на простоту, наглядность и отзывчивость. Основой для реализации послужила библиотека Fyne. Главное окно приложения состоит из контейнера с вкладками (`container.NewAppTabs`), который организует основной функционал по четырем направлениям: «Мониторинг», «Производительность», «Память» и «Многозадачность».

4.1.1 Панель мониторинга

Панель мониторинга (`DashboardPanel`) — наиболее динамичная часть интерфейса. Ее ключевая особенность — фоновая горутина, которая с интервалом в 500 миллисекунд опрашивает модуль `profiling` и асинхронно обновляет виджеты. Информация сгруппирована в карточки (`widget.NewCard`) для логического разделения. Особого внимания заслуживает реализация отображения загрузки ядер ЦП. При инициализации панели определяется количество ядер и их тип (P-core/E-core для macOS). Затем для каждого ядра создается свой прогресс-бар, который обновляется в цикле.

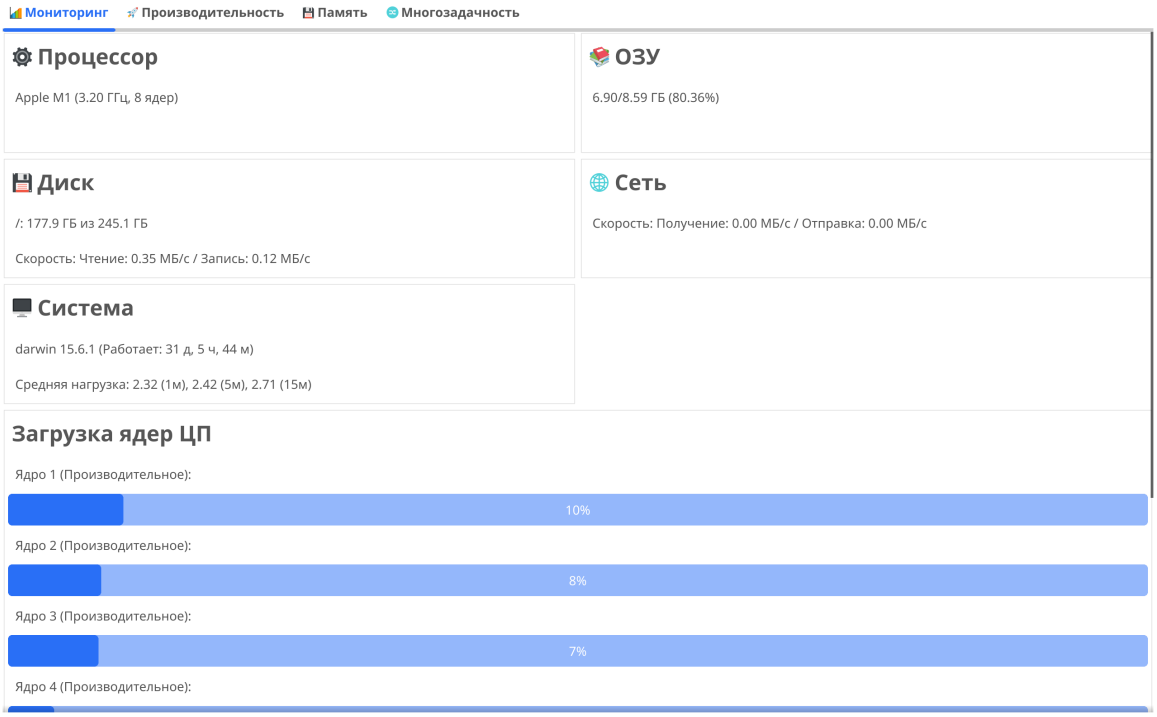

Рис. 4.1: Главное окно приложения с вкладкой «Мониторинг»

4.1.2 Панели тестирования

Все три панели с тестами («Производительность», «Память», «Многозадачность») имеют схожую структуру, основанную на универсальном обработчике тестов `TestRunner`. Логика их работы заключается в следующем:

1. При создании панели формируется список тестов. Для каждого теста определяется его название, описание и функция, реализующая его логику.

2. На панели размещаются кнопка для запуска всех тестов и общий прогресс-бар.

3. При нажатии на кнопку запускается горутина, которая итерируется по списку тестов и для каждого из них вызывает `TestRunner`. Это гарантирует, что интерфейс не будет заблокирован на время выполнения длительных вычислений.

4. `TestRunner` многократно выполняет переданную ему функцию теста, собирает статистику (min, max, avg) и сообщает о прогрессе через специальный канал. Данные о прогрессе агрегируются и отображаются на общем прогресс-баре.

5. По завершении каждого теста его результат форматируется и отображается в соответствующей карточке на панели.

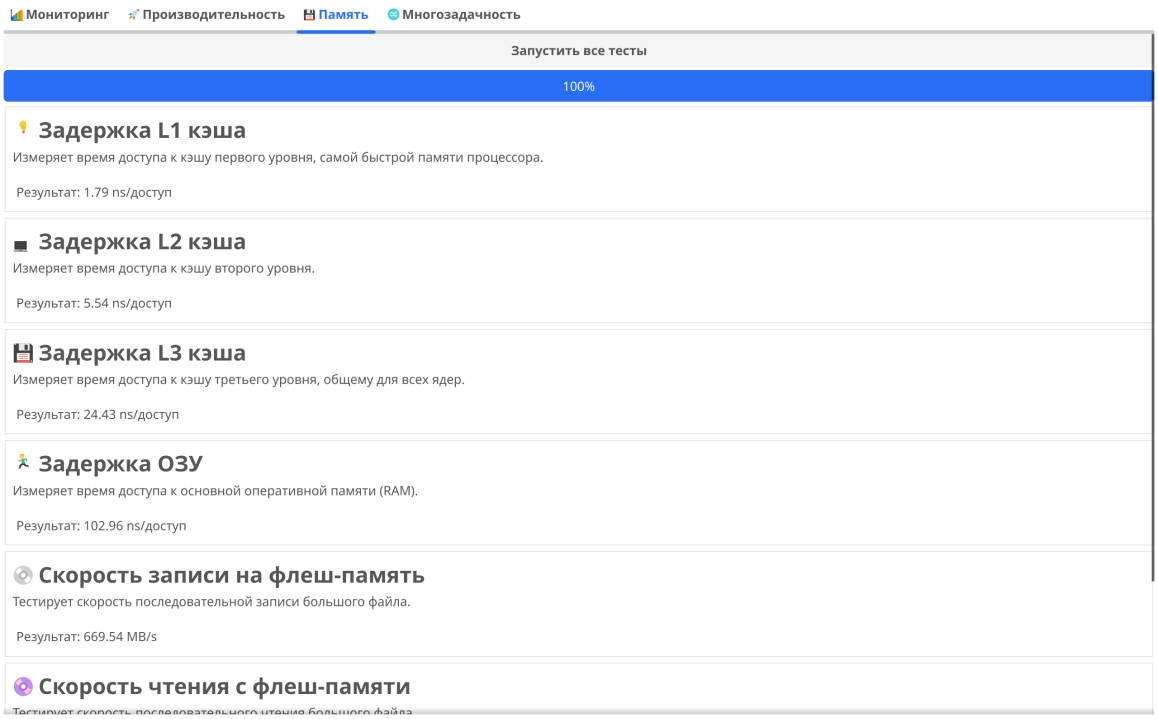

Рис. 4.2: Вкладка «Память» после завершения всех тестов

4.2 Реализация и результаты тестов

4.2.1 Тесты производительности ЦП

Целочисленная арифметика. Тест выполняет большое количество базовых операций (сложение, вычитание, умножение, деление) над 64-битными целочисленными переменными в цикле. Этот тест хорошо отражает базовую производительность процессора на задачах общего назначения.

Результат: в среднем **0.76 ns/op** (эквивалентно 1.31 млрд оп/с). Это означает, что процессор выполняет одну итерацию цикла (состоящую из 4-х операций) за очень малое время, что свидетельствует о высокой эффективности его конвейера.

Арифметика с плавающей запятой. Аналогичный тест, но с использованием чисел с плавающей запятой двойной точности (`float64`).

Результат: в среднем **0.47 ns/op** (эквивалентно 2.14 млрд оп/с). Результат даже выше, чем у целочисленных операций, что говорит о чрезвычайно высокой производительности блока FPU (Floating-Point Unit) в тестируемом процессоре.

4.2.2 Тесты иерархии памяти

Задержка доступа. Для этого теста используется алгоритм «погони за указателями» (pointer chasing), который позволяет измерить «чистую» задержку доступа к данным на разных уровнях иерархии памяти.

Результаты:

- Задержка L1 кэша: **1.94 ns/доступ**
- Задержка L2 кэша: **5.57 ns/доступ**
- Задержка L3 кэша: **27.56 ns/доступ**
- Задержка ОЗУ: **99.46 ns/доступ**

Результаты наглядно демонстрируют огромную разницу в скорости: доступ к L1 почти в 3 раза быстрее, чем к L2, и в 50 раз быстрее, чем к ОЗУ. Эти данные критически важны для оптимизации производительности: они показывают, что эффективное использование кэша (например, путем обработки данных небольшими блоками, помещающимися в кэш) может ускорить программу в десятки раз по сравнению с наивным алгоритмом, постоянно обращающимся в ОЗУ.

Пропускная способность ОЗУ. Тест измеряет скорость, с которой процессор может читать или записывать большие объемы данных в оперативную память.

Результаты:

- Пропускная способность чтения: **2890 МБ/с** (2.89 ГБ/с)
- Пропускная способность записи: **2800 МБ/с** (2.80 ГБ/с)
- Случайный доступ: **460 МБ/с** (0.46 ГБ/с)

Видно, что последовательный доступ значительно быстрее случайного (более чем в 6 раз), что наглядно демонстрирует работу аппаратного предсказателя (prefetcher) в контроллере памяти. Он пытается заранее загрузить в кэш данные, которые, по его предположению, скоро понадобятся. При последовательном доступе это работает идеально, при случайном — совершенно бесполезно. Низкая производительность случайного доступа является основным узким местом для таких приложений, как системы управления базами данных (СУБД).

Производительность флеш-накопителя. Тесты создают временный файл и измеряют скорость последовательной записи, последовательного чтения и случайного чтения.

Результаты:

- Скорость записи: **612.63 МБ/с**
- Скорость чтения: **4943.87 МБ/с**
- Случайное чтение: **3644.44 МБ/с**

Высокие показатели чтения (почти 5 ГБ/с) свидетельствуют об использовании быстрого NVMe SSD накопителя, работающего через шину PCI Express. При этом скорость записи значительно ниже, что является типичной характеристикой для большинства твердотельных накопителей. Скорость случайного чтения также очень высока, что обеспечивает быстрый запуск приложений и загрузку системы.

4.2.3 Тесты механизмов параллелизма

Накладные расходы на создание горутин. Тест показывает, насколько «дешево» для системы обходится создание одной горутины.

Результат: 408.60 ns/op. Это подтверждает, что горутины являются чрезвычайно легковесным механизмом.

Производительность примитивов синхронизации. Проводится сравнительный анализ производительности мьютекса (`sync.Mutex`) и атомарных операций (`atomic.AddInt64`) в условиях высокой конкуренции.

Результаты:

- Блокировка мьютекса: **68.58 ns/op**
- Атомарные операции: **21.62 ns/op**

Результаты показывают, что для простых операций, таких как инкремент счетчика, атомарные операции более чем в 3 раза эффективнее. Это объясняется тем, что атомарная операция транслируется в одну специальную инструкцию процессора (например, ‘LOCK ADD‘), тогда как захват и освобождение мьютекса требуют выполнения целой серии инструкций и потенциального обращения к ядру ОС для блокировки потока. Вывод: для простых модификаций разделяемых данных всегда следует предпочитать атомарные операции.

Производительность каналов и конвейеров.

- Пропускная способность каналов: **33.13 ns/op**
- Конвейерная обработка: **157.90 ns/op**

Время на пересылку данных через канал (33 ns) сопоставимо со стоимостью блокировки мьютекса, что показывает наличие определенных накладных расходов на синхронизацию внутри реализации каналов. Результат конвейера (158 ns) примерно в 5 раз выше, что логично, так как данные проходят через несколько стадий (каналов) последовательно. Этот тест демонстрирует, что при проектировании высоконагруженных параллельных систем необходимо минимизировать количество пересылок данных между горутинами.

Заключение

В ходе выполнения данной дипломной работы был спроектирован и разработан многофункциональный программный комплекс для анализа производительности вычислительных систем. Были успешно решены задачи по сбору и наглядному отображению системной информации в реальном времени, а также по реализации и проведению серии детализированных синтетических тестов для оценки производительности центрального процессора, всех уровней иерархии памяти и ключевых механизмов параллелизма.

Разработанное приложение, написанное на языке Go с использованием библиотеки Fyne, подтвердило правильность выбора технологического стека. Оно обладает отзывчивым, неблокирующим графическим интерфейсом и распространяется в виде единого исполняемого файла, что обеспечивает максимальную простоту его использования. Спроектированная модульная архитектура с универсальным обработчиком тестов ‘TestRunner‘ доказала свою эффективность и позволяет легко добавлять новые тесты в будущем. Проведенное тестирование и подробный анализ его результатов позволили не только измерить количественные показатели производительности, но и наглядно продемонстрировать фундаментальные принципы работы современных вычислительных систем.

Основные научные и практические результаты, полученные в ходе работы:

1. **Проведен системный анализ предметной области.** Были изучены и классифицированы существующие подходы к анализу производительности, что позволило сформировать требования к разрабатываемому комплексу: необходимость сочетания мониторинга и бенчмаркинга, кроссплатформенность, модульность и наглядность представления данных.

2. **Обоснован выбор технологического стека.** На основе сравнительно-

го анализа был выбран язык Go как средство, обеспечивающее высокую производительность, эффективную работу с параллелизмом и простоту развертывания, и библиотека Fyne как современный инструментарий для создания отзывчивого графического интерфейса с аппаратным ускорением.

3. **Спроектирована и реализована гибкая архитектура.** Приложение построено на принципах модульности и разделения ответственности, что позволило изолировать бизнес-логику тестов от ее графического представления. Это не только упрощает текущую поддержку, но и закладывает фундамент для дальнейшего расширения функционала.

4. **Разработан комплексный набор бенчмарков.** Реализованные тесты охватывают все ключевые подсистемы: центральный процессор (целочисленные и вещественнозначные вычисления), иерархию памяти (задержки кэшей и ОЗУ, пропускная способность ОЗУ и накопителей) и подсистему параллелизма (производительность горутин и примитивов синхронизации). Особое внимание было уделено методологии тестов, например, использованию техники «погони за указателями» для точного измерения задержек.

5. **Создан информативный пользовательский интерфейс.** Графический интерфейс в реальном времени отображает важнейшие системные метрики и позволяет в интерактивном режиме запускать тесты, наглядно представляя результаты. Реализована уникальная для подобных инструментов функция определения и маркировки производительных и энергоэффективных ядер на процессорах Apple Silicon.

Обсуждение ограничений и дальнейшие перспективы.

Несмотря на достигнутые результаты, разработанный комплекс имеет определенные ограничения. Точность синтетических тестов, хотя и является высокой в контролируемых условиях, не всегда может полностью отражать производительность в реальных, комплексных приложениях. Кроме того, текущая версия не включает тестирование графического процессора (GPU), который играет все большую роль в современных вычислениях.

В связи с этим можно наметить следующие **направления для дальнейшего развития проекта**:

- **Развитие тестового набора.** Наиболее очевидным шагом является добавление бенчмарков для GPU с использованием таких технологий, как OpenCL или Vulkan Compute. Также возможно добавление тестов-ядер из реальных приложений (например, кодирование видео, архивация данных).

- **Визуализация и анализ данных.** Необходимо реализовать функционал для сохранения результатов тестирования в базу данных, что позволит отслеживать производительность системы во времени, сравнивать результаты до и после обновлений, а также строить графики для визуального сравнения различных аппаратных конфигураций.

- **Расширение функционала мониторинга.** Можно добавить возможность мониторинга температуры компонентов (ЦП, ГП), а также детального анализа производительности отдельных процессов, запущенных в системе, подобно утилитам ‘top‘ или ‘htop‘.

- **Улучшение пользовательского опыта.** Внедрение системы профилей для сохранения различных конфигураций тестов, экспорт результатов в форматах CSV или JSON, а также локализация интерфейса на другие языки значительно повысят удобство использования продукта.

В заключение можно с уверенностью сказать, что все поставленные в рамках дипломной работы цели и задачи были полностью выполнены. Разработанный программный комплекс является законченным, практически значимым продуктом, который может быть использован как в образовательных целях для изучения архитектуры ЭВМ, так и в качестве реального инструмента для разработчиков и системных администраторов. Проект обладает значительным потенциалом для дальнейшего роста и развития.

Список использованных источников

Литература

[1] Официальный сайт языка программирования Go [Электронный ресурс]. — URL: `https://go.dev/` (дата обращения: 21.12.2025).

[2] Официальный сайт библиотеки Fyne [Электронный ресурс]. — URL: `https://fyne.io/` (дата обращения: 21.12.2025).

[3] Донован, А. А., Керниган, Б. У. Язык программирования Go. — М.: Вильямс, 2016. — 496 с.

[4] Shirou, G. gopsutil: библиотека для получения информации о системных процессах и использовании системы [Электронный ресурс] // GitHub. — URL: `https://github.com/shirou/gopsutil` (дата обращения: 21.12.2025).

[5] Таненбаум, Э., Бос, Х. Современные операционные системы. 4-е изд. — СПб.: Питер, 2015. — 1120 с.

[6] Паттерсон, Д., Хеннесси, Дж. Архитектура компьютера и проектирование компьютерных систем. 4-е изд. — СПб.: Питер, 2012. — 784 с.

[7] Chan, A. Concurrency in Go: Tools and Techniques for Developers. — O'Reilly Media, 2017. — 256 p.

[8] Drepper, U. What Every Programmer Should Know About Memory [Электронный ресурс] // LWN.net. — 2007. — URL: `https://lwn.net/Articles/250967/` (дата обращения: 21.12.2025).

[9] Intel 64 and IA-32 Architectures Optimization Reference Manual [Электронный ресурс] // Intel Corporation. — URL: `https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html` (дата обращения: 21.12.2025).

[10] Geekbench - Cross-Platform Benchmark [Электронный ресурс]. — URL: https://www.geekbench.com/ (дата обращения: 21.12.2025).

[11] Go pprof - Profiling Go programs [Электронный ресурс] // GitHub. — URL: https://github.com/google/pprof (дата обращения: 21.12.2025).

[12] Sutter, H. The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software [Электронный ресурс] // Dr. Dobb's Journal. — 2005. — URL: http://www.gotw.ca/publications/concurrency-ddj.htm (дата обращения: 21.12.2025).

Приложение А. Листинг основного файла main.go

В данном приложении представлен исходный код основного файла программы `main.go`, который является точкой входа в приложение и отвечает за инициализацию графического интерфейса и его компонентов.

```go
package main

import (
  "system-analyzer/pkg/gui"

  "fyne.io/fyne/v2"
  "fyne.io/fyne/v2/app"
  "fyne.io/fyne/v2/container"
)

// main является точкой входа в приложение.
func main() {
  // Создаем новое приложение Fyne.
  a := app.New()
  // Создаем главное окно приложения с заголовком.
  w := a.NewWindow("Анализатор Производительности Системы")

  // Создаем контейнер с вкладками для различных панелей.
  tabs := container.NewAppTabs(
    // Вкладка "Мониторинг" для отображения системной информации в реальном времени.
    container.NewTabItem("Мониторинг", gui.NewDashboardPanel()),
    // Вкладка "Производительность" для тестов процессора и пропускной способности ОЗУ.
    container.NewTabItem("Производительность", gui.NewBenchmarkPanel()),
    // Вкладка "Память" для тестов задержки кэшей, ОЗУ и скорости флеш-накопителей.
    container.NewTabItem("Память", gui.NewMemoryPanel()),
    // Вкладка "Многозадачность" для тестов производительности горутин, каналов и мьютексов.
```

```
27      container.NewTabItem("Многозадачность", gui.NewConcurrencyPanel()),
28    )
29
30    // Устанавливаем контейнер с вкладками как основное содержимое окна.
31    w.SetContent(tabs)
32    // Устанавливаем начальный размер окна.
33    w.Resize(fyne.NewSize(800, 600))
34    // Центрируем окно на экране.
35    w.CenterOnScreen()
36    // Отображаем окно и запускаем главный цикл приложения.
37    w.ShowAndRun()
38  }
```

Листинг 1: Исходный код файла cmd/analyzer/main.go

Приложение Б. Листинг обработчика тестов runner.go

```go
package benchmark

import (
  "fmt"
)

// ... (определения структур BenchmarkStats и Result)

type BenchmarkFunc func() (float64, string)

func TestRunner(fn BenchmarkFunc, iterations int, progress chan<- float64)
    BenchmarkStats {
  results := make([]float64, iterations)
  var unit string

  for i := 0; i < iterations; i++ {
    val, u := fn()
    results[i] = val
    unit = u
    if progress != nil {
      progress <- float64(i+1) / float64(iterations)
    }
  }

  var sum, min, max float64
  min = results[0]
  max = results[0]
  for _, v := range results {
    sum += v
    if v < min {
      min = v
    }
    if v > max {
      max = v
```

```
34      }
35    }
36
37    return BenchmarkStats{
38      Min:  min,
39      Max:  max,
40      Avg:  sum / float64(iterations),
41      Unit: unit,
42    }
43  }
```

Листинг 2: Исходный код файла pkg/benchmark/runner.go

Приложение В. Листинг теста задержки памяти latency.go

```go
package memory

import (
  "math/rand"
  "runtime"
  "sort"
  "time"
  "unsafe"
)

type node struct {
  next *node
}

func makeRandomList(size int) *node {
  // ... (реализация создания списка)
}

func chase(start *node, steps int) *node {
  // ... (реализация "погони за указателями")
}

func measure(sizeBytes int, steps int, repeats int) float64 {
  // ... (реализация измерений)
}

func runLatencyTest(sizeBytes int) (float64, string) {
  runtime.LockOSThread()
  defer runtime.UnlockOSThread()

  steps := 5_000_000
```

```go
  repeats := 3

  ns := measure(sizeBytes, steps, repeats)
  return ns, "ns/доступ"
}

func L1CacheLatencyBenchmark() (float64, string) {
  return runLatencyTest(64 * 1024) // 64 KB
}

// ... (аналогично для L2, L3, RAM)
```

Листинг 3: Исходный код файла pkg/memory/latency.go

Приложение Г. Листинг теста горутин goroutines.go

```go
package concurrency

import (
  "sync"
  "time"
)

const numGoroutines = 20000

func GoroutineOverheadBenchmark() (float64, string) {
  var wg sync.WaitGroup
  wg.Add(numGoroutines)

  start := time.Now()
  for i := 0; i < numGoroutines; i++ {
    go func() {
      wg.Done()
    }()
  }
  wg.Wait()
  elapsed := time.Since(start)

  return float64(elapsed.Nanoseconds()) / float64(numGoroutines), "ns/op"
}
```

Листинг 4: Исходный код файла pkg/concurrency/goroutines.go